Human Verification: `Containers.Dirichlet`

The Dirichlet Tensor $(\mathbf{Cont}, \otimes, y)$

Machine-generated from `lean/Dirichlet.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Dirichlet` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

The file defines the **Dirichlet tensor** $C \otimes D$ (shapes $S \times T$, positions $P\,s \times Q\,t$), proves it is a monoidal structure on **Cont** (all coherences by `rfl`), and discusses when $[\![-]\!]$ is (or is not) strong monoidal for $\otimes$. A uniformity morphism $C \otimes D \to C \triangleleft D$ is defined, and shown to be a proper mono (not an iso).

Contents

1 Overview **2**

2 The tensor and its unit **2**
2.1 Definition: `Container.dirichlet` . . . 2
2.2 Definition: `Container.y` . . . 2

3 Bifunctoriality **3**
3.1 Definition: $\texttt{Container.dir}_2$ . . . 3
3.2 Theorem: $\texttt{Container.dir}_2\texttt{_id}$ . . . 3
3.3 Theorem: $\texttt{Container.dir}_2\texttt{_comp}$ . . . 3

4 Structural isomorphisms **3**
4.1 Definition: `Container.dirAssociator` . . . 3
4.2 Definition: `Container.dirLeftUnitor` . . . 4
4.3 Definition: `Container.dirRightUnitor` . . . 4

5 Naturality and coherence **5**
5.1 Theorem: `Container.dirAssociator_naturality` . . . 5
5.2 Theorem: `Container.dirLeftUnitor_naturality` . . . 5
5.3 Theorem: `Container.dirRightUnitor_naturality` . . . 5
5.4 Theorem: `Container.dirPentagon` . . . 6
5.5 Theorem: `Container.dirTriangle` . . . 6
5.6 Definition: `contDirichletMonoidal` . . . 6

6 Extension and the Dirichlet tensor **7**
6.1 Theorem: `Container.dirichletExt_normalForm` . . . 7

7 The uniformity morphism $C \otimes D \to C \triangleleft D$ **7**
7.1 Definition: `Container.dirToSeq` 7
7.2 Helper definitions: `Container.ySq`, `Container.two`, `Container.seqShapeSnd` . 8
7.3 Theorem: `Container.dirToSeq_not_surjective` 8

1 Overview

For containers $C = S \triangleleft P$ and $D = T \triangleleft Q$, the Dirichlet tensor is:

$$C \otimes D = (S \times T) \triangleleft (\lambda\,(s,t).\ P\,s \times Q\,t).$$

Contrast the categorical product:

$$C \times D = (S \times T) \triangleleft (\lambda\,(s,t).\ P\,s \sqcup Q\,t).$$

The shapes are the same; only the positions differ — product vs. sum.

The key asymmetry is in extensions:

$$[\![C \times D]\!]X = \Sigma_{(s,t)},\ X^{P\,s \sqcup Q\,t} \cong [\![C]\!]X \times [\![D]\!]X \qquad \text{(pointwise, by } y^{A+B} \cong y^A \times y^B\text{)},$$
$$[\![C \otimes D]\!]X = \Sigma_{(s,t)},\ X^{P\,s \times Q\,t} = \Sigma_{(s,t)},\ (X^{P\,s})^{Q\,t} \qquad \text{(not pointwise; } y^{A\times B} \text{ does not split)}.$$

2 The tensor and its unit

2.1 Definition: `Container.dirichlet`

Definition 2.1 (Dirichlet tensor)**.** For containers C and D, the **Dirichlet tensor** $C \otimes D$ has:

- shapes: $S_C \times S_D$,
- positions at (s,t): $P_C(s) \times P_D(t)$.

Lean.
```
def Container.dirichlet (C D : Container) : Container where
  Shape := C.Shape * D.Shape
  Pos   := fun st ⇒ C.Pos st.1 * D.Pos st.2
```

Haskell.
```
dirichlet :: Container s p → Container t q → Container (s, t) (p, q)
-- Shape = (s, t);  Pos (s, t) = (p, q)
```

2.2 Definition: `Container.y`

Definition 2.2 (Unit for $\otimes$)**.** The unit for the Dirichlet tensor is $y = \mathbf{1} \triangleleft (\lambda\ast.\,\mathbf{1})$ — the same container as I, the unit for $\triangleleft$. In the Dirichlet setting it is read as the *representable* y^1.

$$y \otimes C \text{ has shapes } \mathbf{1} \times S \cong S \text{ and positions } \mathbf{1} \times P\,s \cong P\,s.$$

Lean.
```
abbrev Container.y : Container := Container.I
```

Haskell.
```
-- y = I = the container ((), const ())
-- Unit for both otimes and triangleleft.
containerY :: Container () ()
containerY = Container () (const ())
```

3 Bifunctoriality

3.1 Definition: `Container.dir₂`

Definition 3.1 (Bifunctorial action of $\otimes$)**.** For morphisms $\mu : C \Rightarrow C'$ and $\nu : D \Rightarrow D'$, the action $\mathtt{dir}_2\,\mu\,\nu : C \otimes D \Rightarrow C' \otimes D'$ is *componentwise*:

- on shapes: $(s,t) \mapsto (\mu(s), \nu(t))$,
- on positions (backward): (s,t), $(p,q) \mapsto (\mu_s^\sharp(p), \nu_t^\sharp(q))$.

Lean.

```
def Container.dir2 {C C' D D' : Container}
    (mu : ContainerMorphism C C') (nu : ContainerMorphism D D') :
    ContainerMorphism (C |*| D) (C' |*| D') where
  onShapes := fun st ⇒ (mu.onShapes st.1, nu.onShapes st.2)
  onPos    := fun st pq ⇒ (mu.onPos st.1 pq.1, nu.onPos st.2 pq.2)
```

Haskell.

```
dir2 :: Morph c p c' p' → Morph d q d' q'
     → Morph (c `Dir` d) (p, q) (c' `Dir` d') (p', q')
dir2 mu nu = Morph
  { onShapes = \(s, t) → (onShapes mu s, onShapes nu t)
  , onPos    = \(s, t) (p, q) → (onPos mu s p, onPos nu t q) }
```

Fidelity note. Unlike $\triangleleft$, the Dirichlet morphism action has no outer-into-inner threading: both coordinates are treated independently. This makes the functor laws hold by `rfl` — a purely definitional computation.

3.2 Theorem: `Container.dir₂_id`

Theorem 3.2 (Dirichlet bifunctoriality: identity)**.** $\mathtt{dir}_2\,(\mathrm{id}_C)\,(\mathrm{id}_D) = \mathrm{id}_{C \otimes D}$. Holds by `rfl`.

Lean.

```
theorem Container.dir2_id (C D : Container) :
    Container.dir2 (ContainerMorphism.id C) (ContainerMorphism.id D)
      = ContainerMorphism.id (C |*| D) := rfl
```

3.3 Theorem: `Container.dir₂_comp`

Theorem 3.3 (Dirichlet bifunctoriality: interchange)**.** $\mathtt{dir}_2\,(\mu \cdot \mu')\,(\nu \cdot \nu') = \mathtt{dir}_2\,\mu\,\nu \cdot \mathtt{dir}_2\,\mu'\,\nu'$. Holds by `rfl`.

Lean.

```
theorem Container.dir2_comp {C C' C'' D D' D'' : Container}
    (mu : ContainerMorphism C C') (mu' : ContainerMorphism C' C'')
    (nu : ContainerMorphism D D') (nu' : ContainerMorphism D' D'') :
    Container.dir2 (mu.comp mu') (nu.comp nu')
      = (Container.dir2 mu nu).comp (Container.dir2 mu' nu') := rfl
```

4 Structural isomorphisms

4.1 Definition: `Container.dirAssociator`

Definition 4.1 (Dirichlet associator)**.** The isomorphism $(C \otimes D) \otimes E \cong C \otimes (D \otimes E)$:

- Shapes: Prod-reassociation $((s,t),u) \mapsto (s,(t,u))$.

- Positions (backward): also Prod-reassociation, running backwards: given $(p, (q, r))$, produce $((p, q), r)$ (and vice versa).

Both round trips hold by `rfl` (η for Prod).

Lean.
```
def Container.dirAssociator (C D E : Container) :
    ContainerIso ((C |*| D) |*| E) (C |*| (D |*| E)) where
  hom :=
    { onShapes := fun x ⇒ (x.1.1, (x.1.2, x.2))
      onPos    := fun _ p ⇒ ((p.1, p.2.1), p.2.2) }
  inv :=
    { onShapes := fun x ⇒ ((x.1, x.2.1), x.2.2)
      onPos    := fun _ p ⇒ (p.1.1, (p.1.2, p.2)) }
  hom_inv := rfl
  inv_hom := rfl
```

Haskell.
```
dirAssociator ::
  ContainerIso (Dir (Dir c d) e) (Dir c (Dir d e))
-- Shapes: ((s,t),u) <-> (s,(t,u))
-- Positions (backward): ((p,q),r) <-> (p,(q,r))
```

4.2 Definition: `Container.dirLeftUnitor`

Definition 4.2 (Dirichlet left unitor)**.** The isomorphism $y \otimes C \cong C$:

- Forward on shapes: $(*, s) \mapsto s$.
- Forward on positions (backward): $p \mapsto (*, p)$.
- Backward on shapes: $s \mapsto (*, s)$.
- Backward on positions (backward): $(*, p) \mapsto p$.

Both round trips hold by `rfl`.

Lean.
```
def Container.dirLeftUnitor (C : Container) :
    ContainerIso (Container.y |*| C) C where
  hom :=
    { onShapes := fun x ⇒ x.2
      onPos    := fun _ p ⇒ ((), p) }
  inv :=
    { onShapes := fun s ⇒ ((), s)
      onPos    := fun _ p ⇒ p.2 }
  hom_inv := rfl
  inv_hom := rfl
```

Haskell.
```
dirLeftUnitor :: ContainerIso (Dir YContainer c) c
-- fwd shapes: ((), s) -> s
-- bwd positions: p -> ((), p)
```

4.3 Definition: `Container.dirRightUnitor`

Definition 4.3 (Dirichlet right unitor)**.** The isomorphism $C \otimes y \cong C$:

- Forward on shapes: $(s, *) \mapsto s$.
- Forward on positions (backward): $p \mapsto (p, *)$.

Both round trips hold by `rfl`.

Lean.
```
def Container.dirRightUnitor (C : Container) :
    ContainerIso (C |*| Container.y) C where
  hom :=
    { onShapes := fun x ⇒ x.1
      onPos    := fun _ p ⇒ (p, ()) }
  inv :=
    { onShapes := fun s ⇒ (s, ())
      onPos    := fun _ p ⇒ p.1 }
  hom_inv := rfl
  inv_hom := rfl
```

5 Naturality and coherence

5.1 Theorem: `Container.dirAssociator_naturality`

Theorem 5.1 (Naturality of Dirichlet associator)**.**

$$(\mathtt{dir}_2\,(\mathtt{dir}_2\,f_1\,f_2)\,f_3)\cdot\alpha^{\otimes}_{Y_1,Y_2,Y_3} \;=\; \alpha^{\otimes}_{X_1,X_2,X_3}\cdot\mathtt{dir}_2\,f_1\,(\mathtt{dir}_2\,f_2\,f_3).$$

Holds by `rfl`.

Lean.
```
theorem Container.dirAssociator_naturality
    {X1 Y1 X2 Y2 X3 Y3 : Container}
    (f1 : ContainerMorphism X1 Y1)
    (f2 : ContainerMorphism X2 Y2)
    (f3 : ContainerMorphism X3 Y3) :
    (Container.dir2 (Container.dir2 f1 f2) f3).comp
        (Container.dirAssociator Y1 Y2 Y3).hom
      = (Container.dirAssociator X1 X2 X3).hom.comp
          (Container.dir2 f1 (Container.dir2 f2 f3)) := rfl
```

5.2 Theorem: `Container.dirLeftUnitor_naturality`

Theorem 5.2 (Naturality of Dirichlet left unitor)**.** $(\mathtt{dir}_2\,(\mathrm{id}_y)\,f)\cdot\lambda^{\otimes}_Y = \lambda^{\otimes}_X\cdot f$. Holds by `rfl`.

Lean.
```
theorem Container.dirLeftUnitor_naturality {X Y : Container}
    (f : ContainerMorphism X Y) :
    (Container.dir2 (ContainerMorphism.id Container.y) f).comp
        (Container.dirLeftUnitor Y).hom
      = (Container.dirLeftUnitor X).hom.comp f := rfl
```

5.3 Theorem: `Container.dirRightUnitor_naturality`

Theorem 5.3 (Naturality of Dirichlet right unitor)**.** $(\mathtt{dir}_2\,f\,(\mathrm{id}_y))\cdot\rho^{\otimes}_Y = \rho^{\otimes}_X\cdot f$. Holds by `rfl`.

Lean.
```
theorem Container.dirRightUnitor_naturality {X Y : Container}
    (f : ContainerMorphism X Y) :
    (Container.dir2 f (ContainerMorphism.id Container.y)).comp
        (Container.dirRightUnitor Y).hom
      = (Container.dirRightUnitor X).hom.comp f := rfl
```

5.4 Theorem: `Container.dirPentagon`

Theorem 5.4 (Pentagon for $\otimes$)**.** The two re-association paths of $((C_1 \otimes C_2) \otimes C_3) \otimes C_4 \to C_1 \otimes (C_2 \otimes (C_3 \otimes C_4))$ agree. Holds by `rfl`.

Lean.

```
theorem Container.dirPentagon (C1 C2 C3 C4 : Container) :
    ((Container.dir2 (Container.dirAssociator C1 C2 C3).hom
          (ContainerMorphism.id C4)).comp
        (Container.dirAssociator C1 (C2 |*| C3) C4).hom).comp
        (Container.dir2 (ContainerMorphism.id C1)
          (Container.dirAssociator C2 C3 C4).hom)
      = (Container.dirAssociator (C1 |*| C2) C3 C4).hom.comp
          (Container.dirAssociator C1 C2 (C3 |*| C4)).hom := rfl
```

5.5 Theorem: `Container.dirTriangle`

Theorem 5.5 (Triangle for $\otimes$)**.** $\alpha^{\otimes}_{C_1,y,C_2} \cdot (\mathrm{id}_{C_1} \otimes \lambda^{\otimes}_{C_2}) = \rho^{\otimes}_{C_1} \otimes \mathrm{id}_{C_2}$. Holds by `rfl`.

Lean.

```
theorem Container.dirTriangle (C1 C2 : Container) :
    (Container.dirAssociator C1 Container.y C2).hom.comp
        (Container.dir2 (ContainerMorphism.id C1)
          (Container.dirLeftUnitor C2).hom)
      = Container.dir2 (Container.dirRightUnitor C1).hom
          (ContainerMorphism.id C2) := rfl
```

5.6 Definition: `contDirichletMonoidal`

Definition 5.6 ($(\mathbf{Cont}, \otimes, y)$ is a monoidal category)**.** A named `def`: the Dirichlet tensor $\otimes$ with morphism action $\mathtt{dir}_2$, unit y, and the structural isomorphisms above satisfy all monoidal-category axioms. Every law holds by `rfl`.

Lean.

```
@[reducible] def contDirichletMonoidal : MonoidalCategory Container
   where
  tensorObj          := Container.dirichlet
  tensorHom          := Container.dir2
  tensorHom_id       := Container.dir2_id
  tensorHom_comp     := fun f1 g1 f2 g2 ⇒ Container.dir2_comp f1 g1 f2
      g2
  tensorUnit         := Container.y
  associator         := fun X Y Z ⇒ (Container.dirAssociator X Y Z).
     toCatIso
  leftUnitor         := fun X ⇒ (Container.dirLeftUnitor X).toCatIso
  rightUnitor        := fun X ⇒ (Container.dirRightUnitor X).toCatIso
  associator_naturality  :=
    fun f1 f2 f3 ⇒ Container.dirAssociator_naturality f1 f2 f3
  leftUnitor_naturality  := fun f ⇒ Container.
     dirLeftUnitor_naturality f
  rightUnitor_naturality := fun f ⇒ Container.
     dirRightUnitor_naturality f
  pentagon           := Container.dirPentagon
  triangle           := Container.dirTriangle
```

Haskell.

```
contDirichletMonoidal :: MonoidalCategoryDef ContainerMorph
-- tensor = dirichlet, unit = y, all laws by rfl.
```

6 Extension and the Dirichlet tensor

6.1 Theorem: `Container.dirichletExt_normalForm`

Theorem 6.1 (Normal form for the Dirichlet extension)**.** By definition (i.e. by `rfl`):

$$[\![C \otimes D]\!]X \;=\; \Sigma_{(s,t):S\times T},\ (P\,s \times Q\,t \to X).$$

This is a *computation / unfolding rule*, not a strong-monoidality statement. The right-hand side mentions the *presentation* S, T, P, Q, not the functors $[\![C]\!]$ and $[\![D]\!]$. The correct intrinsic account is that $[\![-]\!]$ is *strong* (not strict) monoidal for $\otimes$, the target tensor being the Day convolution of $(\mathbf{Set}, \times, \mathbf{1})$ (Niu–Spivak, Prop. 3.79).

Lean.

```
theorem Container.dirichletExt_normalForm
    (C D : Container) (X : Type) :
    Ext (C |*| D) X
      = ((st : C.Shape * D.Shape) * (C.Pos st.1 * D.Pos st.2 → X)) :=
        rfl
```

Haskell.

```
-- By definition:
-- Ext (dirichlet c d) x  =  ((s,t), (p,q) → x)
-- This is NOT the same as Ext c x `tensor` Ext d x for any pointwise
   tensor.
```

Fidelity note. Three of the four monoidal structures give genuine (non-identity) comparison isomorphisms for $[\![-]\!]$: $[\![G \triangleleft F]\!]X \cong [\![G]\!]([\![F]\!]X)$, $[\![C + D]\!]X \cong [\![C]\!]X \sqcup [\![D]\!]X$, and $[\![C \times D]\!]X \cong [\![C]\!]X \times [\![D]\!]X$. For $\otimes$ there is no such pointwise comparison, witnessed by $y^3 \otimes y^3 = y^9$ while any pointwise tensor would give y^6.

7 The uniformity morphism $C \otimes D \to C \triangleleft D$

7.1 Definition: `Container.dirToSeq`

Definition 7.1 (Uniformity morphism)**.** The canonical map $C \otimes D \Rightarrow C \triangleleft D$ sends the shape (s, t) to $(s, \lambda -.\ t)$ (the constant inner-shape assignment):

$$(s, t) \;\longmapsto\; (s,\ \lambda p.\ t).$$

On positions: the identity, since the fibres agree definitionally: $\Sigma\, p : P\,s,\ Q((\lambda -.\ t)\,p) = P\,s \times Q\,t$.

Lean.

```
def Container.dirToSeq (C D : Container) :
    ContainerMorphism (C |*| D) (C seq D) where
  onShapes := fun st ⇒ ⟨st.1, fun _ ⇒ st.2⟩
  onPos    := fun _ pq ⇒ (pq.1, pq.2)
```

Haskell.

```
dirToSeq :: ContainerMorphism (c `Dir` d) (c `Seq` d)
dirToSeq = Morph
  { onShapes = \(s, t) → (s, const t)  -- constant inner assignment
  , onPos    = \_ (p, q) → (p, q) }
```

7.2 Helper definitions: `Container.ySq`, `Container.two`, `Container.seqShapeSnd`

Definition 7.2 (Auxiliary containers for the counterexample)**.**

- $y^2 = \mathbf{1} \triangleleft (\lambda\, *\,.\, \mathrm{Bool})$: one shape, two positions.
- $2 = \mathrm{Bool} \triangleleft (\lambda -.\, \emptyset)$: two shapes, no positions.
- `seqShapeSnd`: extracts the inner-shape function from a shape of $y^2 \triangleleft 2$, viewed as a plain function Bool $\to$ Bool. Defined to enable `congrArg` in the counterexample.

Lean.

```
def Container.ySq : Container where
  Shape := Unit
  Pos   := fun _ ⇒ Bool

def Container.two : Container where
  Shape := Bool
  Pos   := fun _ ⇒ Empty

def Container.seqShapeSnd
    (z : (Container.ySq seq Container.two).Shape) : Bool → Bool := z
       .2
```

Haskell.

```
ySq :: Container () Bool    -- one shape, two positions
ySq = Container { shape = (), pos = const False {- placeholder -} }

two :: Container Bool Void  -- two shapes, no positions
two = Container { shapes = [True, False], pos = const absurd }
```

7.3 Theorem: `Container.dirToSeq_not_surjective`

Theorem 7.3 (The uniformity morphism is not an isomorphism)**.** The shape map of `dirToSeq` for $C = y^2$, $D = 2$ is Bool $\ni t \mapsto (*, \lambda -.\, t)$, which hits only *constant* functions Bool $\to$ Bool. The identity function id : Bool $\to$ Bool is not in the image. Therefore `dirToSeq` is not surjective on shapes, hence not an isomorphism.

Concretely, $y^2 \otimes 2$ has 2 shapes while $y^2 \triangleleft 2$ has 4, so the extensions differ: $[\![y^2 \otimes 2]\!]X$ has 2 elements and $[\![y^2 \triangleleft 2]\!]X$ has 4. Thus $\otimes \neq \triangleleft$.

Lean.

```
theorem Container.dirToSeq_not_surjective :
    not (Exists fun st : (Container.ySq |*| Container.two).Shape ⇒
        (Container.dirToSeq Container.ySq Container.two).onShapes st
          = (⟨(), fun b ⇒ b⟩ : (Container.ySq seq Container.two).
             Shape))
```

Haskell.

```
-- The image of dirToSeq on shapes is { ((), const t) | t :: Bool }.
-- The shape ((), id) of (ySq `Seq` two) is not in this image,
-- since id True = True /= False = id False but const t True = const t
     False.
-- So dirToSeq is not surjective, hence not an iso.
```

Fidelity note. The proof works by contradiction: assuming a preimage st exists, `congrArg seqShapeSnd` extracts the equation $\lambda -.\, st.2 = \mathrm{id}$ on Bool; applying at true gives $st.2 = \mathrm{true}$ and at false gives $st.2 = \mathrm{false}$, a contradiction.